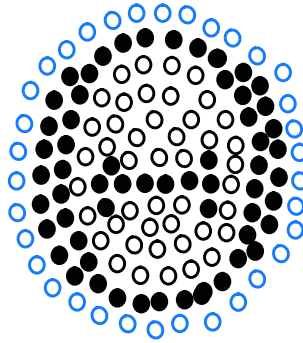




Contents

4.1	Experimental Studies	151
4.1.1	Moving Beyond Experimental Analysis	154
4.2	The Seven Functions Used in This Book	156
4.2.1	Comparing Growth Rates	163
4.3	Asymptotic Analysis	164
4.3.1	The “Big-Oh” Notation	164
4.3.2	Comparative Analysis	168
4.3.3	Examples of Algorithm Analysis	170
4.4	Simple Justification Techniques	178
4.4.1	By Example	178
4.4.2	The “Contra” Attack	178
4.4.3	Induction and Loop Invariants	179
4.5	Exercises	182

In a classic story, the famous mathematician Archimedes was asked to determine if a golden crown commissioned by the king was indeed pure gold, and not part silver, as an informant had claimed. Archimedes discovered a way to perform this analysis while stepping into a bath. He noted that water spilled out of the bath in proportion to the amount of him that went in. Realizing the implications of this fact, he immediately got out of the bath and ran naked through the city shouting, “Eureka, eureka!” for he had discovered an analysis tool (displacement), which, when combined with a simple scale, could determine if the king’s new crown was good or not. That is, Archimedes could dip the crown and an equal-weight amount of gold into a bowl of water to see if they both displaced the same amount. This discovery was unfortunate for the goldsmith, however, for when Archimedes did his analysis, the crown displaced more water than an equal-weight lump of pure gold, indicating that the crown was not, in fact, pure gold.

In this book, we are interested in the design of “good” data structures and algorithms. Simply put, a **data structure** is a systematic way of organizing and accessing data, and an **algorithm** is a step-by-step procedure for performing some task in a finite amount of time. These concepts are central to computing, but to be able to classify some data structures and algorithms as “good,” we must have precise ways of analyzing them.

The primary analysis tool we will use in this book involves characterizing the running times of algorithms and data structure operations, with space usage also being of interest. Running time is a natural measure of “goodness,” since time is a precious resource—computer solutions should run as fast as possible. In general, the running time of an algorithm or data structure operation increases with the input size, although it may also vary for different inputs of the same size. Also, the running time is affected by the hardware environment (e.g., the processor, clock rate, memory, disk) and software environment (e.g., the operating system, programming language) in which the algorithm is implemented and executed. All other factors being equal, the running time of the same algorithm on the same input data will be smaller if the computer has, say, a much faster processor or if the implementation is done in a program compiled into native machine code instead of an interpreted implementation run on a virtual machine. We begin this chapter by discussing tools for performing experimental studies, yet also limitations to the use of experiments as a primary means for evaluating algorithm efficiency.

Focusing on running time as a primary measure of goodness requires that we be able to use a few mathematical tools. In spite of the possible variations that come from different environmental factors, we would like to focus on the relationship between the running time of an algorithm and the size of its input. We are interested in characterizing an algorithm’s running time as a function of the input size. But what is the proper way of measuring it? In this chapter, we “roll up our sleeves” and develop a mathematical way of analyzing algorithms.

4.1 Experimental Studies

One way to study the efficiency of an algorithm is to implement it and experiment by running the program on various test inputs while recording the time spent during each execution. A simple mechanism for collecting such running times in Java is based on use of the `currentTimeMillis` method of the `System` class. That method reports the number of milliseconds that have passed since a benchmark time known as the epoch (January 1, 1970 UTC). It is not that we are directly interested in the time since the epoch; the key is that if we record the time immediately before executing the algorithm and then immediately after, we can measure the *elapsed* time of an algorithm's execution by computing the difference of those times. A typical way to automate this process is shown in Code Fragment 4.1.

```
1 long startTime = System.currentTimeMillis();           // record the starting time
2 /* (run the algorithm) */
3 long endTime = System.currentTimeMillis();           // record the ending time
4 long elapsed = endTime - startTime;                   // compute the elapsed time
```

Code Fragment 4.1: Typical approach for timing an algorithm in Java.

Measuring elapsed time in this fashion provides a reasonable reflection of an algorithm's efficiency; for extremely quick operations, Java provides a method, `nanoTime`, that measures in nanoseconds rather than milliseconds.

Because we are interested in the general dependence of running time on the size and structure of the input, we should perform independent experiments on many different test inputs of various sizes. We can then visualize the results by plotting the performance of each run of the algorithm as a point with x -coordinate equal to the input size, n , and y -coordinate equal to the running time, t . Such a visualization provides some intuition regarding the relationship between problem size and execution time for the algorithm. This may be followed by a statistical analysis that seeks to fit the best function of the input size to the experimental data. To be meaningful, this analysis requires that we choose good sample inputs and test enough of them to be able to make sound statistical claims about the algorithm's running time.

However, the measured times reported by both methods `currentTimeMillis` and `nanoTime` will vary greatly from machine to machine, and may likely vary from trial to trial, even on the same machine. This is because many processes share use of a computer's *central processing unit* (or *CPU*) and memory system; therefore, the elapsed time will depend on what other processes are running on the computer when a test is performed. While the precise running time may not be dependable, experiments are quite useful when comparing the efficiency of two or more algorithms, so long as they gathered under similar circumstances.

As a tangible example of experimental analysis, we consider two algorithms for constructing long strings in Java. Our goal will be to have a method, with a calling signature such as `repeat('*', 40)`, that produces a string composed of 40 asterisks: `"*****"`.

The first algorithm we consider performs repeated string concatenation, based on the `+` operator. It is implemented as method `repeat1` in Code Fragment 4.2. The second algorithm relies on Java's `StringBuilder` class (see Section 1.3), and is implemented as method `repeat2` in Code Fragment 4.2.

```

1  /** Uses repeated concatenation to compose a String with n copies of character c. */
2  public static String repeat1(char c, int n) {
3      String answer = "";
4      for (int j=0; j < n; j++)
5          answer += c;
6      return answer;
7  }
8
9  /** Uses StringBuilder to compose a String with n copies of character c. */
10 public static String repeat2(char c, int n) {
11     StringBuilder sb = new StringBuilder();
12     for (int j=0; j < n; j++)
13         sb.append(c);
14     return sb.toString();
15 }
```

Code Fragment 4.2: Two algorithms for composing a string of repeated characters.

As an experiment, we used `System.currentTimeMillis()`, in the style of Code Fragment 4.1, to measure the efficiency of both `repeat1` and `repeat2` for very large strings. We executed trials to compose strings of increasing lengths to explore the relationship between the running time and the string length. The results of our experiments are shown in Table 4.1 and charted on a log-log scale in Figure 4.1.

<i>n</i>	repeat1 (in ms)	repeat2 (in ms)
50,000	2,884	1
100,000	7,437	1
200,000	39,158	2
400,000	170,173	3
800,000	690,836	7
1,600,000	2,874,968	13
3,200,000	12,809,631	28
6,400,000	59,594,275	58
12,800,000	265,696,421	135

Table 4.1: Results of timing experiment on the methods from Code Fragment 4.2.

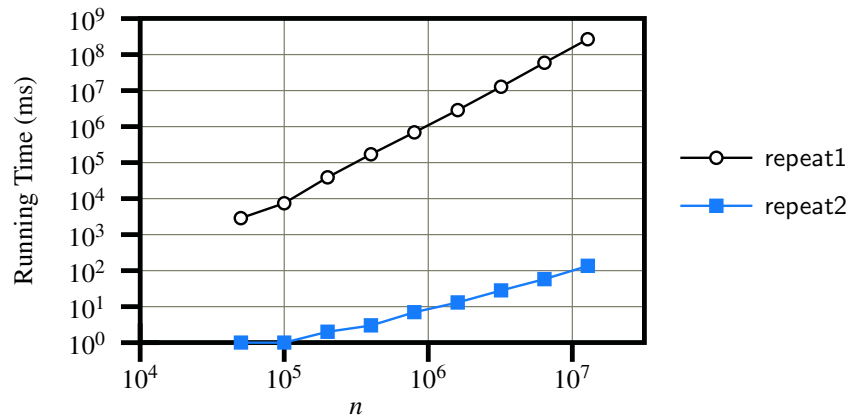


Figure 4.1: Chart of the results of the timing experiment from Code Fragment 4.2, displayed on a log-log scale. The divergent slopes demonstrate an order of magnitude difference in the growth of the running times.

The most striking outcome of these experiments is how much faster the repeat2 algorithm is relative to repeat1. While repeat1 is already taking more than 3 *days* to compose a string of 12.8 million characters, repeat2 is able to do the same in a fraction of a *second*. We also see some interesting trends in how the running times of the algorithms each depend upon the size of n . As the value of n is doubled, the running time of repeat1 typically increases more than fourfold, while the running time of repeat2 approximately doubles.

Challenges of Experimental Analysis

While experimental studies of running times are valuable, especially when fine-tuning production-quality code, there are three major limitations to their use for algorithm analysis:

- Experimental running times of two algorithms are difficult to directly compare unless the experiments are performed in the same hardware and software environments.
- Experiments can be done only on a limited set of test inputs; hence, they leave out the running times of inputs not included in the experiment (and these inputs may be important).
- An algorithm must be fully implemented in order to execute it to study its running time experimentally.

This last requirement is the most serious drawback to the use of experimental studies. At early stages of design, when considering a choice of data structures or algorithms, it would be foolish to spend a significant amount of time implementing an approach that could easily be deemed inferior by a higher-level analysis.

4.1.1 Moving Beyond Experimental Analysis

Our goal is to develop an approach to analyzing the efficiency of algorithms that:

1. Allows us to evaluate the relative efficiency of any two algorithms in a way that is independent of the hardware and software environment.
2. Is performed by studying a high-level description of the algorithm without need for implementation.
3. Takes into account all possible inputs.

Counting Primitive Operations

To analyze the running time of an algorithm without performing experiments, we perform an analysis directly on a high-level description of the algorithm (either in the form of an actual code fragment, or language-independent pseudocode). We define a set of *primitive operations* such as the following:

- Assigning a value to a variable
- Following an object reference
- Performing an arithmetic operation (for example, adding two numbers)
- Comparing two numbers
- Accessing a single element of an array by index
- Calling a method
- Returning from a method

Formally, a primitive operation corresponds to a low-level instruction with an execution time that is constant. Ideally, this might be the type of basic operation that is executed by the hardware, although many of our primitive operations may be translated to a small number of instructions. Instead of trying to determine the specific execution time of each primitive operation, we will simply count how many primitive operations are executed, and use this number t as a measure of the running time of the algorithm.

This operation count will correlate to an actual running time in a specific computer, for each primitive operation corresponds to a constant number of instructions, and there are only a fixed number of primitive operations. The implicit assumption in this approach is that the running times of different primitive operations will be fairly similar. Thus, the number, t , of primitive operations an algorithm performs will be proportional to the actual running time of that algorithm.

Measuring Operations as a Function of Input Size

To capture the order of growth of an algorithm's running time, we will associate, with each algorithm, a function $f(n)$ that characterizes the number of primitive operations that are performed as a function of the input size n . Section 4.2 will introduce the seven most common functions that arise, and Section 4.3 will introduce a mathematical framework for comparing functions to each other.

Focusing on the Worst-Case Input

An algorithm may run faster on some inputs than it does on others of the same size. Thus, we may wish to express the running time of an algorithm as the function of the input size obtained by taking the average over all possible inputs of the same size. Unfortunately, such an **average-case** analysis is typically quite challenging. It requires us to define a probability distribution on the set of inputs, which is often a difficult task. Figure 4.2 schematically shows how, depending on the input distribution, the running time of an algorithm can be anywhere between the worst-case time and the best-case time. For example, what if inputs are really only of types “A” or “D”?

An average-case analysis usually requires that we calculate expected running times based on a given input distribution, which usually involves sophisticated probability theory. Therefore, for the remainder of this book, unless we specify otherwise, we will characterize running times in terms of the **worst case**, as a function of the input size, n , of the algorithm.

Worst-case analysis is much easier than average-case analysis, as it requires only the ability to identify the worst-case input, which is often simple. Also, this approach typically leads to better algorithms. Making the standard of success for an algorithm to perform well in the worst case necessarily requires that it will do well on **every** input. That is, designing for the worst case leads to stronger algorithmic “muscles,” much like a track star who always practices by running up an incline.

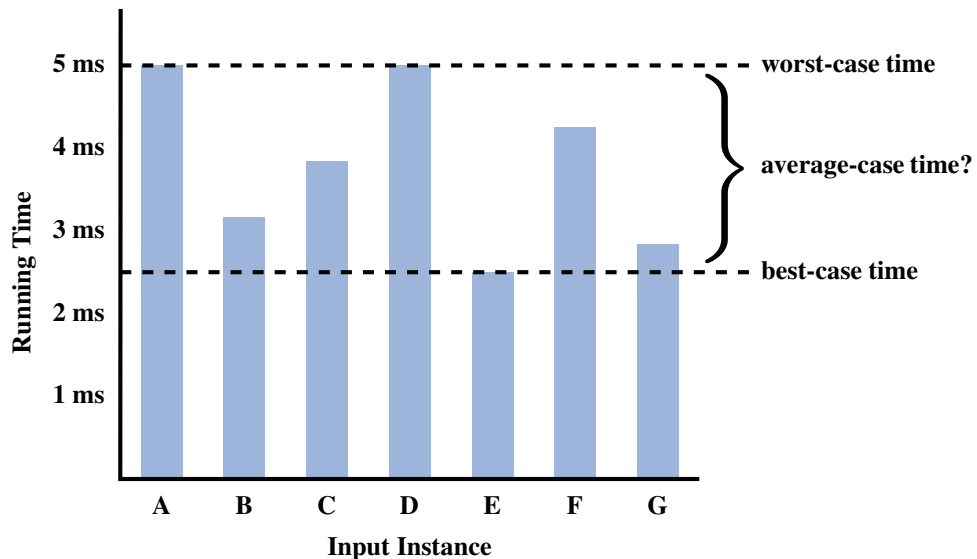


Figure 4.2: The difference between best-case and worst-case time. Each bar represents the running time of some algorithm on a different possible input.